

Lista de Exercícios: Estruturas de Dados Elementares*

PCC104 – Projeto e Análise de Algoritmos
DECOM – UFOP

3 de junho de 2026

Instruções. Esta lista cobre **arrays**, **matrizes**, **pilhas**, **filas**, **listas duplamente encadeadas** e **tabelas hash**. As questões 1 a 16 são teóricas (análise, demonstrações, projeto algorítmico); as questões 17 a 20 envolvem implementação em Python. Em todas as questões, suponha indexação a partir de 1 (estilo CLRS), salvo menção contrária. Quando solicitada análise assintótica, use notação O , Θ ou Ω , conforme o caso, e justifique a escolha. Marcadores indicam dificuldade: (★) introdutória, (★★) intermediária, (★★★) desafiadora.

Parte I – Arrays e Matrizes

- (★) Considere um array $A[1..n]$ de inteiros armazenado em memória contígua.
 - Justifique por que o acesso $A[i]$, para qualquer $1 \leq i \leq n$, é realizado em tempo $\Theta(1)$. Explícite a expressão de endereçamento utilizada.
 - Mostre que, se A está **não ordenado**, a busca por um valor x qualquer leva tempo $\Theta(n)$ no pior caso e $\Theta(n)$ no caso médio (sob hipótese de distribuição uniforme da posição de x).
 - Se A está **ordenado** em ordem crescente, descreva um algoritmo de busca em $O(\log n)$ e justifique informalmente sua corretude e complexidade.
- (★★) Seja $A[1..n, 1..n]$ uma matriz armazenada em *row-major order*. Considere os dois trechos abaixo, ambos calculando $\sum_{i,j} A[i, j]$:

Trecho A	Trecho B
para $i = 1$ até n	para $j = 1$ até n
para $j = 1$ até n	para $i = 1$ até n
$s \leftarrow s + A[i, j]$	$s \leftarrow s + A[i, j]$

 - Ambos têm a mesma complexidade assintótica. Qual?
 - Apesar disso, na prática um deles é tipicamente mais rápido. Qual e por quê? Explique em termos de **localidade espacial** e linhas de cache.
 - Como sua resposta mudaria se a matriz fosse armazenada em *column-major* (como em Fortran ou Julia)?
- (★★) **Array dinâmico.** Considere um array que suporta a operação $\text{APPEND}(x)$. Quando a capacidade c é atingida, alocamos um novo array de capacidade $2c$, copiamos os elementos e liberamos o antigo.
 - Mostre que, no pior caso, uma única operação APPEND custa $\Theta(n)$.
 - Mostre que, em uma sequência de n operações APPEND a partir de um array vazio, o custo **total** é $\Theta(n)$, e portanto o custo **amortizado** por operação é $\Theta(1)$. (Use o argumento agregado.)
 - Por que a estratégia de crescer somando uma constante (e.g., $c \leftarrow c + 10$) leva a custo amortizado $\Theta(n)$ por operação? Justifique.

Parte II – Pilhas

- (★) (Adaptado de CLRS 10.1-1) Seja uma pilha S implementada em um array $S[1..6]$, inicialmente vazia. Mostre o conteúdo de S e o valor do atributo $S.top$ **após cada operação** da sequência:

$\text{PUSH}(S, 4), \text{PUSH}(S, 1), \text{PUSH}(S, 3), \text{POP}(S), \text{PUSH}(S, 8), \text{POP}(S).$

Em seguida, indique o resultado retornado por uma operação POP adicional.

- (★★) (CLRS 10.1-2) Explique como implementar **duas pilhas** S_1 e S_2 em um único array $A[1..n]$ de modo que nenhuma das pilhas estoure desde que o número *total* de elementos nas duas seja, no máximo, n . As

*Material baseado em Cormen, Leiserson, Rivest e Stein (CLRS), *Introduction to Algorithms*, 4ª ed., MIT Press, 2022, capítulos 10 e 11.

operações PUSH e POP devem rodar em $O(1)$. Apresente pseudocódigo e justifique a corretude da detecção de overflow/underflow.

6. (★★) **Verificação de sequência alcançável.** Dada uma sequência de entrada $1, 2, \dots, n$ que é empilhada em ordem em uma pilha (intercalando PUSH e POP arbitrariamente), e dada uma permutação $\pi = \pi_1, \pi_2, \dots, \pi_n$, descreva um algoritmo em $O(n)$ que decide se π pode ser a sequência de saída produzida pelos POPS. Justifique a complexidade..

Parte III – Filas

7. (★) (Adaptado de CLRS 10.1-3) Seja uma fila Q implementada em um array circular $Q[1..6]$, inicialmente vazia. Mostre o conteúdo de Q e os atributos $Q.head$ e $Q.tail$ após cada operação da sequência:

ENQUEUE($Q, 4$), ENQUEUE($Q, 1$), ENQUEUE($Q, 3$), DEQUEUE(Q), ENQUEUE($Q, 8$), DEQUEUE(Q).

8. (★★) (CLRS 10.1-5) Uma **deque** (*double-ended queue*) é uma estrutura que suporta inserção e remoção em ambas as extremidades. Apresente um esquema baseado em array circular para implementar deque com as quatro operações PUSHFRONT, PUSHBACK, POPFRONT, POPBACK todas em tempo $O(1)$. Especifique invariantes e condições de *overflow* e *underflow*.
9. (★★) **Fila com duas pilhas.** Mostre como simular uma fila usando duas pilhas S_1 e S_2 (apenas operações PUSH, POP e teste de vazio).
- Descreva o invariante mantido entre S_1 e S_2 .
 - Qual o custo de pior caso de uma operação DEQUEUE? E o custo **amortizado** por operação em uma sequência de n operações? Justifique usando o método agregado.

Parte IV – Listas Duplamente Encadeadas

10. (★★) (CLRS 10.2-1 generalizado) Considere uma **lista simplesmente encadeada** L (sem ponteiro para o último nó).
- A operação INSERT, que insere um elemento no início de L , pode ser implementada em $O(1)$? Justifique.
 - A operação DELETE(L, x), dado um *ponteiro* x para o nó a remover, pode ser feita em $O(1)$? Compare com o caso de listas **duplamente** encadeadas e explique a diferença.
 - Mostre um caso (em uma lista *simplesmente* encadeada) em que DELETE pode ser feito em $O(1)$ usando um truque com cópia de campo. Quando esse truque *falha*?
11. (★★) **Sentinelas.** Seja L uma lista duplamente encadeada circular com sentinela $L.nil$ (CLRS 10.2, Seção *Sentinelas*). Reescreva os procedimentos LIST-INSERT e LIST-DELETE usando a sentinela. Em seguida, compare-os com as versões sem sentinela:
- Quantas instruções de teste de condição (**se**) cada versão precisa?
 - Em que situação o uso de sentinela pode ser **desvantajoso**?
12. (★★) **Concatenação.** Sejam L_1 e L_2 duas listas duplamente encadeadas com sentinelas, contendo conjuntos *disjuntos* de elementos. Apresente um algoritmo que concatene L_2 ao final de L_1 em tempo $O(1)$, descrevendo todas as atualizações de ponteiros necessárias e o estado final dos sentinelas.

Parte V – Tabelas Hash

13. (★) **Endereçamento direto vs. hash.**
- Descreva quando uma **tabela de endereçamento direto** (*direct-address table*) é uma escolha apropriada. Cite as operações SEARCH, INSERT e DELETE e suas complexidades.
 - Suponha que o universo de chaves seja $U = \{0, 1, \dots, 2^{32} - 1\}$, mas que apenas $n = 10^5$ chaves estão de fato armazenadas. Por que o endereçamento direto é **inviável** nesse cenário? Quanto de memória ele consumiria (em ordem de grandeza)?
 - Explique como uma tabela hash resolve esse problema e onde surge a possibilidade de **colisões**.
14. (★★) **Análise sob encadeamento.** Considere uma tabela hash $T[0..m-1]$ com resolução de colisões por **encadeamento separado**, n chaves armazenadas e fator de carga $\alpha = n/m$. Assuma a **Hipótese da Hashing Uniforme Simples (SUHA)**.
- Mostre que o número esperado de comparações em uma busca **mal-sucedida** é $\Theta(1 + \alpha)$.
 - Mostre que o número esperado de comparações em uma busca **bem-sucedida** é $\Theta(1 + \alpha)$.
 - Conclua: se $n = O(m)$, qual é o tempo esperado das operações SEARCH, INSERT e DELETE?
15. (★★) **Funções hash.** Considere a função de hash pelo método da divisão $h(k) = k \bmod m$.
- Por que escolhas $m = 2^p$ são geralmente ruins? Mostre o que acontece se k é uma cadeia de bits e $m = 2^p$ (qual parte de k é efetivamente usada?).

- (b) Por que escolhas m próximo de uma *potência de 10* também podem ser ruins se as chaves provêm de identificadores decimais?
- (c) Descreva o **método da multiplicação** $h(k) = \lfloor m(kA \bmod 1) \rfloor$, com $0 < A < 1$. Quais escolhas de A Knuth sugere e por quê esse método é menos sensível à escolha de m ?
16. (★★) **Endereçamento aberto.** Considere uma tabela hash com endereçamento aberto, fator de carga $\alpha = n/m < 1$, e **hashing uniforme** (toda permutação de sondagem é igualmente provável).
- (a) Mostre que o número esperado de sondagens em uma **busca mal-sucedida** é, no máximo, $1/(1 - \alpha)$.
- (b) Conclua que o número esperado de sondagens em uma INSERT é também no máximo $1/(1 - \alpha)$.
- (c) Compare **sondagem linear**, **sondagem quadrática** e **duplo hashing**. Defina e diferencie *clustering primário* e *clustering secundário*, indicando qual estratégia sofre de qual fenômeno.

Parte VI – Implementação (Python)

Observação. As questões a seguir devem ser entregues como arquivos `.py` acompanhados de testes unitários (`unittest` ou `pytest`). Siga as convenções do CLRS para nomes de procedimentos e mantenha as complexidades assintóticas declaradas.

17. (★★) **Pilha com mínimo em $O(1)$.** Implemente uma classe `MinStack` que suporta as operações `PUSH( $x$ )`, `POP()`, `TOP()` e `MIN()`, todas em tempo $O(1)$ no **pior caso**.
- (a) Descreva a ideia da solução (use uma estrutura auxiliar).
- (b) Argumente por que o espaço total ainda é $O(n)$.
- (c) Inclua casos de teste com pelo menos três sequências distintas, incluindo uma com elementos repetidos.
18. (★★) **Lista duplamente encadeada com sentinela.** Implemente uma classe `DoublyLinkedList` com um nó sentinela *nil* e os procedimentos:
- `LIST-SEARCH( $L, k$ )` – busca em $O(n)$.
 - `LIST-INSERT( $L, x$ )` – inserção no início em $O(1)$.
 - `LIST-DELETE( $L, x$ )` – remoção em $O(1)$ dado o ponteiro x .
- Compare seu código com a versão *sem* sentinela: quantas linhas de `if` foram eliminadas?
19. (★★) **Tabela hash com encadeamento.** Implemente `HashTableChaining` usando **encadeamento separado** (cada *slot* aponta para uma lista encadeada).
- (a) Use a função hash pelo método da divisão $h(k) = k \bmod m$, com m primo, recebido no construtor.
- (b) Implemente `INSERT`, `SEARCH`, `DELETE` e um método `load_factor()` que retorna $\alpha = n/m$.
- (c) Inclua um método `rehash()` que duplica m (para o próximo primo $\geq 2m$) e reinsere todas as chaves quando $\alpha > 1.0$. Justifique brevemente, em comentário, por que o custo amortizado por `INSERT` permanece $\Theta(1)$.
20. (★★★) **LRU Cache.** Implemente uma classe `LRUCache` com capacidade fixa C que suporta:
- `get(key)` – retorna o valor associado à chave em tempo $O(1)$, ou -1 se ausente.
 - `put(key, value)` – insere ou atualiza o par em tempo $O(1)$. Quando a cache excede C , remove o item **menos recentemente usado**.

Dica: combine uma tabela hash (mapa de chaves para nós) com uma lista duplamente encadeada (ordem de uso). Justifique, no relatório, por que cada operação é $O(1)$ no pior caso, e descreva o invariante que conecta as duas estruturas.